Parallel Matrix Multiplication using Multiprocessing

A Course Project Report

Submitted in the partial fulfilment of the completion of the course

23CS302PC303- High Performance Computing

In III Year II, Semester B. Tech- CSE

Submitted by

Srichala Ankam

2303A52435

Under the supervision of the faculty

Dr. K Deepa

Assistant Professor, SoCS & AI

SR University

Academic Year: ***2025-2026***

Department of Computer Science and Artificial Intelligence

SR University

Warangal, Telangana, India.

INDEX

1. INTRODUCTION

High Performance Computing is really important for solving problems on computers. It does this by using computer methods. One of the ideas in High Performance Computing is parallel computing. This means that the computer does calculations at the same time. This helps to make the computer work faster and do a job. This project is, about using computing and seeing how well it works. We are using something called matrix multiplication to test it. Matrix multiplication is an example because it is a big problem that needs a lot of computer power. So we are going to use computing to do matrix multiplication and see what happens.

Matrix multiplication is really important for a lot of things like computer graphics and machine learning. We also use matrix multiplication for image processing and simulations.

Matrix multiplication is usually done one step at a time. We calculate each part of the matrix one after the other.

This way of doing matrix multiplication is not hard to understand. It is easy to make it work. When we have really big matrices this method is not good because it takes a long time to finish and it needs a lot of computer power. Matrix multiplication becomes slow, for matrices.

To get around these problems we can use computing techniques. When we do matrix multiplication in parallel, we break down the work into jobs that do not depend on each other. These jobs can be done at the time using many processors or CPU cores. This really cuts down the time it takes to get the job done and makes it more efficient especially when we are working with a lot of data. Parallel computing techniques and parallel matrix multiplication are very useful, for this.

In this project we implemented matrix multiplication using Pythons multiprocessing module. the multiprocessing module helps create processes that run at the same time. This makes the most of the computer's CPU. Each process gets a part of the work like calculating rows of the resulting matrix.

We combine the results at the end. The multiprocessing module makes parallel matrix multiplication possible. It uses processes to speed up the computation.

The project also includes a comparison between parallel approaches to see how well they perform. We measure how long it takes to execute for matrix sizes to analyse how well parallel computing works. The effectiveness of computing is analysed by looking at these execution times. It is observed that sequential computation does okay for input sizes. However parallel computation gives big performance boosts for matrices. The project shows that parallel computing is really useful when dealing with matrices. Sequential approach is fine for matrices but parallel approach is better, for bigger ones.

In conclusion, it is evident that parallel processing plays an essential role in contemporary computing systems, and computational efficiency can be improved through the distribution of work among several processing units. This assignment has helped gain insight on the utilization of multiprocessing systems in solving problems in real-time and enhancing efficiency.

2. PROBLEM STATEMENT

Multiplication of matrices is a fundamental process which finds application in several different areas including computer science, data analytics, machine learning, and scientific computing. This process consists of performing various arithmetic calculations on two input matrices, to obtain a resultant matrix. Simple as it may sound, the complexity of this operation increases greatly as the size of the input matrices increase.

The conventional serial approach to this problem involves calculation of each element one at a time. While it is easy to implement, this approach becomes slow and highly complex when the matrices are very large. In the current computing scenario, there is an increasing need for more efficient methods of computing larger datasets.

One example of task division in order to break down a complex operation into several simple ones is a parallel computing method. Nevertheless, implementing parallel processing causes additional complications in the form of process management, communication overhead, and synchronization.

In turn, in this research project, it will be necessary to conduct a study on creating an efficient algorithm for performing matrix multiplication in parallel and analyzing its execution speed compared to the sequential method. Thus, the aim is to examine whether parallel computation decreases the time spent on calculations.

The development of the demand was driven by the need for high performance computations in today's applications. There are various practical situations where computations are very complex and often require massive matrix calculations.

It is clear that sequential computation cannot be used when dealing with massive computations since they are computed one-by-one. It is clear that sequential computation cannot be applied for HPC (high performance computations).

In a parallel computing approach, several processors or processor cores are used simultaneously for computing processes. With the implementation of parallel computing by performing parallel matrix multiplications, one learns how to improve computation speed, along with solving the constraints related to parallel computing such as overhead and scalability. The current project has presented a valid case of the application of parallel computing. Some goals of this project are:

1. To understand the concept of matrix multiplication and its computational complexity.
2. To implement the sequential matrix multiplication method using Python.
3. To implement parallel matrix multiplication using the multiprocessing technique.
4. To measure and compare the execution time of sequential and parallel approaches.
5. To analyze the performance of both methods for different matrix sizes.
6. To study the effect of parallel processing overhead on small input sizes.
7. To demonstrate the advantages of parallel computing for large-scale computations.
8. To visualize the performance comparison using graphical representation.

3. METHODOLOGY/PROCEDURE

3.1 Overview of Methodology

The methodology used in this project is centred on the study and comparison of sequential and parallel techniques of matrix multiplication. The aim of the methodology is to measure and compare the performance of both techniques to illustrate the superiority of parallel computing as an approach that cuts down the processing time of complex computations. The whole process was done systematically and in an organized manner.

3.2 Environment Setup

The first step involved establishing the environment through Google Colab. This is an efficient platform for implementing Python codes. Once the environment was established, the required libraries including multiprocessing, time, random, and matplotlib were imported. All these libraries have been widely utilized during the development process since multiprocessing helped with parallel processing, time for performance evaluation, random for generating input values, and matplotlib for visualizing output values.

3.3 Matrix Generation

After finishing with the environmental setup process, it was time for the next task, which was that of choosing the size of the matrix (N) and creating two matrices (A and B) of the same size, namely N by N. In order to make them realistic, their components would be filled with random numbers. When conducting the experiment, the number N was equal to 150.

3.4 Sequential Matrix Multiplication & Execution Time Measurement (Sequential)

Sequential multiplication technique of matrices was then employed. This technique made use of three loops, whereby each loop computed an individual element in the final matrix. Two outer loops went through the rows and columns of the final matrix while the inner loop conducted the multiplication of elements. While simple to execute, this technique takes up much more time when dealing with large matrices. At the end of the computation process, the program's running time was logged using the time module.

3.5 Parallel Matrix Multiplication

After that, the parallel matrix multiplication approach was developed through the use of multiprocessing approach provided by the Python programming language. In this approach, the total computation was subdivided into smaller tasks. For instance, each row of the resultant matrix was handled by an individual process, and thus computations could be executed concurrently. A pool of processes was formed according to the number of the computer's CPU cores. Once the total computation is divided, it would be assigned to individual processes. These processes execute their individual task, after which all the computations are gathered together to form the final matrix.

3.6 Performance Evaluation

The performance of both sequential and parallel approaches was evaluated after implementing them for N = 150. The results obtained indicated that the parallel approach was faster than the sequential approach because several activities were performed at the same time. The performance improvement achieved using the parallel approach was further quantified using speedup, which is the ratio of the execution times of the sequential and parallel approaches.

3.7 Testing

For further analysis, the test was extended into different matrix dimensions. This involves repeating the entire process of matrix creation, computation, timing, parallel computing, and timing with matrix dimensions of N=50, 100, 150, and 200. This will allow us to understand how well both algorithms behave as the input size is increased.

3.8 Graphical Representation

These were then plotted to represent the data. A bar chart was drawn to compare the timings for the sequential and parallel approaches when the size of the matrix (N) was 150. Similarly, a line chart was drawn to compare the performance trend with respect to the varying size of the matrices. This made it abundantly clear that even though there was no substantial difference in performance for small sized matrices, the parallel approach far outshone the sequential approach for large sized matrices.

3.9 Analysis

The overhead caused by the process creation, communication, and synchronization in case of parallel execution has also been identified in this regard. Such overheads affect the performance of small matrix sizes, thereby making the sequential method faster than the parallel method. With an increase in matrix size, the amount of computation increases, which helps the parallel method perform better.

Last but not least, all the results obtained were analysed and recorded. The execution times and graph analyses gave us a good idea about how the two algorithms behaved. The findings from this experiment indicated that breaking down a complex computation problem into several small parts would improve efficiency.

3.10 Summary

Therefore, all steps including the creation of the environment, creation of matrices, sequential algorithm implementation, parallel algorithm implementation, results comparison, and drawing of graphs were systematically performed. The methodology employed was able to accomplish the set aim of comparing sequential and parallel matrices multiplication algorithms and emphasize on the importance of parallel processing in HPC.

4. CODE

4.1 Libraries & Sequential Matrix Multiplication

[2] ✓ 3s

```
import multiprocessing as mp
import time
import random

# --------------------------------
# Generate Matrix
# --------------------------------
def generate_matrix(n):
    return [[random.randint(1, 10) for _ in range(n)] for _ in range(n)]


# ================================
# SEQUENTIAL MATRIX MULTIPLICATION
# ================================
def sequential_multiply(A, B, n):
    result = [[0]*n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            for k in range(n):
                result[i][j] += A[i][k] * B[k][j]

    return result
```

4.2 Parallel Matrix Multiplication

```
# ================================
# PARALLEL MATRIX MULTIPLICATION
# ================================
def compute_row(args):
    A, B, row, n = args
    result_row = []

    for j in range(n):
        sum_val = 0
        for k in range(n):
            sum_val += A[row][k] * B[k][j]
        result_row.append(sum_val)

    return result_row


def parallel_multiply(A, B, n):
    pool = mp.Pool(mp.cpu_count())

    tasks = [(A, B, i, n) for i in range(n)]
    result = pool.map(compute_row, tasks)

    pool.close()
    pool.join()

    return result
```

4.3 Main Execution

```
# ================================
# MAIN EXECUTION
# ================================
if __name__ == "__main__":

    sizes = [50, 100, 150, 200]

    for n in sizes:
        print(f"\nMatrix Size: {n} x {n}")

        A = generate_matrix(n)
        B = generate_matrix(n)

        # -------- Sequential --------
        start = time.time()
        sequential_multiply(A, B, n)
        end = time.time()
        print("Sequential Time:", end - start)

        # -------- Parallel --------
        start = time.time()
        parallel_multiply(A, B, n)
        end = time.time()
        print("Parallel Time:", end - start)
```

4.4 Results & Graphs

[4] ✓ 0s

```
import matplotlib.pyplot as plt

# Your actual results
sizes = [50, 100, 150, 200]

seq_times = [0.010571002960205078, 0.0864877700805664,0.5617117881774902,1.3683276176452637]
par_times = [ 0.03669595718383789, 0.12702417373657227, 0.5194797515869141, 0.78741455078125]

# ========================
# BAR GRAPH (N = 150)
# ========================
index = sizes.index(150)

plt.figure()
plt.bar(["Sequential", "Parallel"], [seq_times[index], par_times[index]])
plt.title("Sequential vs Parallel (N = 150)")
plt.xlabel("Method")
plt.ylabel("Execution Time (seconds)")
plt.show()
```

```
# ========================
# LINE GRAPH (ALL SIZES)
# ========================
plt.figure()
plt.plot(sizes, seq_times, marker='o', label="Sequential")
plt.plot(sizes, par_times, marker='o', label="Parallel")

plt.title("Performance Comparison")
plt.xlabel("Matrix Size")
plt.ylabel("Execution Time (seconds)")
plt.legend()

plt.show()
```

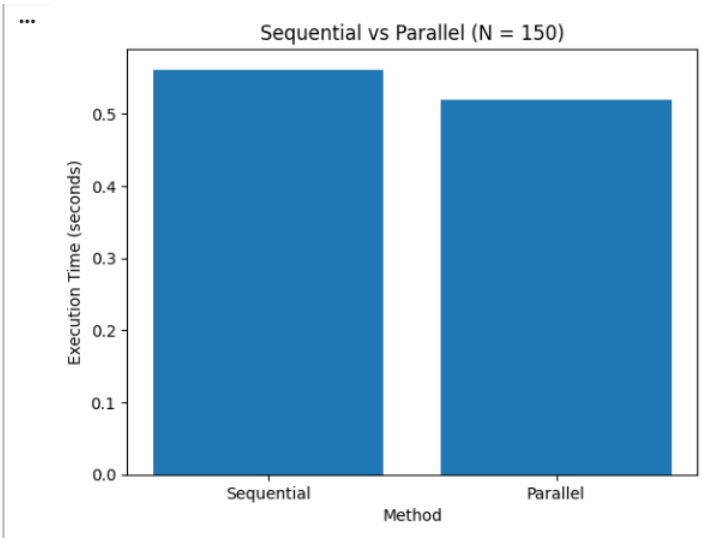
5. OUTPUT SCREENSHOTS

5.1 Results

```
Matrix Size: 50 x 50
Sequential Time: 0.010571002960205078
Parallel Time: 0.03669595718383789

Matrix Size: 100 x 100
Sequential Time: 0.0864877700805664
Parallel Time: 0.12702417373657227

Matrix Size: 150 x 150
Sequential Time: 0.5617117881774902
Parallel Time: 0.5194797515869141

Matrix Size: 200 x 200
Sequential Time: 1.3683276176452637
Parallel Time: 0.78741455078125
```

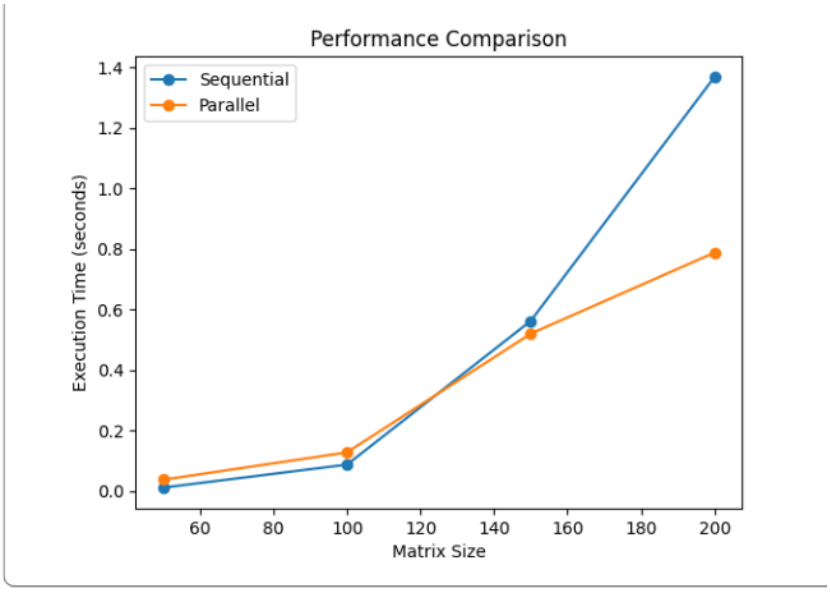
5.2 Visual Representation

- Execution Time Comparison between Sequential and Parallel Execution for Matrix Size 150



It can be observed from the bar graph that the execution time required by the parallel algorithm is less than the execution time required by the sequential algorithm.

- Performance Comparisons for Various Sizes of the Matrix



From the line graph above, it can be seen that the execution time increases as the size of the matrix increases for both the algorithms. But the increase in execution time is smaller for the parallel algorithm.

6. REFERENCES

1. Hennessy, J. L., & Patterson, D. A. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 2019.
2. Quinn, M. J. *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2004.
3. Grama, A., Gupta, A., Karypis, G., & Kumar, V. *Introduction to Parallel Computing*. Addison-Wesley, 2003.
4. Python Software Foundation. *Python 3 Documentation*. https://docs.python.org/3/
5. Python Software Foundation. *Multiprocessing Module Documentation*. https://docs.python.org/3/library/multiprocessing.html
6. Matplotlib Developers. *Matplotlib Documentation*. https://matplotlib.org/stable/contents.html